AI Engineering Concept Map: Raw Material for a Training Schema

Compiled October 23, 2026. Purpose: a dense inventory of concepts, rules of thumb and experiments to hand to Opus, which will turn it into a structured learning/experiment schema.

Provenance legend. [S] = found in web searches during this conversation (sources listed at the end). **[K]** = from Claude's background knowledge (cutoff June 2026); verify before relying on it, since this field moves fast. Numbers marked [S] come mostly from vendor blogs and a few arXiv papers; treat them as rough guides and re-measure on your own data.

---

1. Think like the model (mental models)

1. **Output is a function of context.** If the needed fact, rule or example isn't in the window, the model is guessing. Debug by asking "could a smart stranger answer this from only what I gave it?" [K]
2. **Decision difficulty hierarchy (easiest to hardest).** Closed-set classification with 2-7 labels > extraction from provided text > binary check with a quoted evidence span > pairwise comparison > anchored ordinal rubric > holistic 1-10 score > counting/arithmetic/long-range bookkeeping > judging its own output or its own confidence. Design each layer so it only asks the easy kind of question. [S for binary/scale/pairwise parts, K for the full ordering]
3. **Reduce the entropy of each call.** Constrain the output space (enums, schemas, short answers). One decision per call. Every extra decision bundled in compounds error. [K]
4. **Autoregressive ordering matters.** The model commits to tokens left to right. Put evidence and reasoning before the verdict in the schema; a verdict-first field makes the reasoning a post-hoc rationalization. [K, consistent with S: judges should cite evidence per criterion]
5. **Known judge/model biases:** position bias, verbosity bias (model-dependent, small in many frontier judges), self-preference (favoring own model family), sycophancy (agreeing with the user or with the draft being judged), anchoring on examples. [S for position/verbosity/self-preference; K for sycophancy/anchoring]

6. **Verbalized confidence is weak.** "Confidence: 90%" is poorly calibrated. Better signals: token logprobs (where available), agreement across multiple samples, agreement across different models, or a separate verifier. [K]
7. **Prompt failure vs capability failure.** Before blaming the model, run a ceiling test: give the task to the strongest model with the oracle context and best prompt. If it still fails, it's capability or label ambiguity; if it passes, it's your prompt/context/layer. [K]
8. **More context is not free.** Long contexts degrade attention to the middle, distractors cause errors, and cost/latency rise. Curate; don't dump. [K]
9. **Ambiguity ceiling.** If two expert humans disagree on a label, no model can beat that agreement rate; the task definition is the bottleneck, not the model. [K]
10. **Silent drift.** Model versions, providers and default parameters change under you. Pin versions, log them, and re-run regression suites on change. [S: freeze judge model/rubric/temperature; K: broader]

---

2. Decision design, schemas and taxonomies

- **Binary decomposition / decision trees of yes-no checks.** Binary pass/fail is the most reliable and easiest to audit; scales add run-to-run disagreement; judge-human alignment degrades as rubric granularity increases. Decision trees of binary checks are more stable than holistic scores. Replace "quality 1-5" with 3-6 binary sub-questions. [S]
- **Evidence-before-verdict schema.** Require a quoted span or citation for each criterion; reject verdicts without evidence. [S]
- **Enums with an escape hatch.** Always include "other" and/or "unsure/insufficient information" so the model isn't forced to misclassify. [K]
- **MECE categories.** Mutually exclusive, collectively exhaustive. Overlapping categories are the #1 cause of inconsistent classification. Test: can two annotators disagree on this item because two categories both fit? [K]
- **Observable-criteria definitions.** Define each label by what's literally in the text or output, not by vibes ("contains a claim with no supporting source" beats "seems untrustworthy"). [K]
- **Cardinality limit per call.** Accuracy usually falls as label count rises; keep roughly 7 or fewer per decision, otherwise go hierarchical (coarse then fine classification in two calls). [K, test it: see experiment E3]

- **Boundary few-shots.** Put examples at the decision boundary (near-misses, hard negatives), not just clean prototypical cases. [K]
- **Anchored ordinal scales.** If you must use a scale, define each level with concrete examples; unanchored scales collapse to the 4-5 range. [S]
- **Pairwise vs pointwise; reference-based vs reference-free.** Pairwise is easier for models than absolute scoring; reference-based (compare to a gold answer) beats reference-free when a gold answer exists. Swap order and require the verdict to survive the swap. [S for swap, K for rest]
- **Constrained decoding / structured outputs.** JSON-schema-enforced output guarantees syntax, not correctness. Field order and enum descriptions still affect accuracy. [K]
- **Abstention and escalation paths.** Design what happens when the model says "unsure": route to a stronger model, a human, or a retrieval step. [K]
- **Taxonomy lifecycle.** Open coding (free-form notes on 50-100 real traces) then axial coding (cluster notes into named categories) then iterate until new traces stop producing new failure modes (saturation). Revisit as the product changes. [S]
- **First upstream failure.** In a multi-step trace, label the earliest step that went wrong; downstream errors are usually consequences. [S]
- **Rationale-length smell test.** If the judge's rationale is regularly longer than the rubric, the rubric is under-specified. [S]

---

3. Pipeline and layer architecture

- **Tiered checking.** Deterministic code checks first (regex, schema validation, unit tests, exact match), then cheap binary model gates, then an LLM judge on a sample (reported guidance: 5-20%), then humans on disagreements and edge cases. [S for tiers, K for ordering rationale]
- **Workflows vs agents.** Prefer fixed workflows (prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer) over open-ended agents when the task is well understood; add autonomy only where flexibility pays for the loss of predictability. [K, from Anthropic's published agent guidance; re-read the current version]
- **Routing and cascades.** Cheap model first; escalate on low agreement, low logprob, failed validation or "unsure." Sweep the escalation threshold to trace a cost/accuracy curve. [K]

- **Error compounding.** Per-step accuracies multiply (five steps at 95% is about 77%). Add validation checkpoints between steps and shorten chains. [K]
- **Verifier/generator asymmetry.** Checking an answer is often easier than producing it; exploit with best-of-N sampling plus a verifier, or generate-then-verify loops. [K]
- **Self-consistency / majority vote.** Sample N times and vote; works best for tasks with a discrete answer. Costs N times as much. [K]
- **Jury of small diverse models.** Three smaller models from different families with majority vote beat a single large judge in one study, at much lower cost and with less per-model bias. [S]
- **Independent judge.** Use a judge from a different model family than the generator to reduce self-preference. [S]
- **Retrieval (RAG).** Evaluate retrieval and generation separately (did the right chunk come back? given it, did the model use it?). Hybrid search (keyword + dense), reranking, chunking strategy and query rewriting are the main levers. Contextual chunk enrichment helps. [K]
- **Context engineering.** Decide deliberately what enters the window: instructions, retrieved facts, tool results, memory. Trim or summarize tool outputs, compact long histories, keep stable prefixes for prompt caching. [K]
- **Tool design (agent-computer interface).** Few, clearly named tools; unambiguous parameters; error messages that tell the model how to recover; return only the fields needed. [K]
- **Reliability plumbing.** Timeouts, retries with backoff, idempotent actions, fallbacks to a second provider/model, circuit breakers, rate-limit handling. [K]
- **Cost and latency levers.** Prompt caching, smaller models for easy layers, parallel calls, streaming, batch APIs for offline work, shorter outputs. [K]
- **Guardrails.** Input checks (injection, PII, off-topic) and output checks (schema, policy, grounding). Treat retrieved/tool content as untrusted data, never as instructions. [K]

---

4. Evaluation and judges

- **Error analysis first, metrics second.** Spend the first effort reading real traces, not designing dashboards. Reported guidance: a large share (60-80%) of development time goes to error analysis and evaluation. [S]

- **Failure-category evaluators.** Each named failure category from your taxonomy gets one evaluator: code check if possible, binary LLM judge if the category is semantic and persistent. Don't build judges for one-off bugs. [S]
- **Gold set.** Reported guidance: 200-500 human-labeled traces drawn from real traffic. Label binary with a short note. [S]
- **Measure the judge, not just the app.** Compute judge-vs-human kappa and per-class true positive / true negative rates, not just accuracy. Reported guidance: about 85-90% agreement before gating releases; kappa under about 0.6 is an alarm. [S]
- **Kappa deflation.** Raw agreement overstates quality; in a 2026 large-scale study the gap between exact match and Cohen's kappa was 33-41 points on MT-Bench. [S]
- **Consistency is not validity.** A judge can have test-retest reliability above 0.95 and still show severe position bias. Check bias separately from stability. [S]
- **Judge rankings don't transfer.** Rankings of judge models shifted by up to 14 positions across benchmarks; pick your judge on your gold set. [S]
- **Stack the cheap mitigations.** Position-swapping + chain-of-thought + explicit rubric gave the biggest gain in one April 2026 benchmark; chain-of-thought alone helped universally. [S]
- **Freeze the judge.** Same model, rubric version, low temperature; re-validate when any changes. [S]
- **Don't overfit the judge prompt to the gold set.** Split gold labels into a dev set (to tune the prompt) and a held-out test set (to report agreement). [K]
- **A 100% pass rate means the eval is too easy.** Add harder cases from production failures. [S]
- **Correct for judge error when estimating prevalence.** If a judge has known TPR/TNR, the observed pass rate is a biased estimate of the true rate; there are simple corrections. [K]
- **Offline vs online.** Offline: fixed datasets and regression suites before deploy. Online: sample production traces, monitor drift, feed new failures back into the dataset (a flywheel). [K]
- **Agent evals.** Grade outcomes (end state of the environment) and, separately, trajectories (wrong tools, loops, wasted steps). Use pass@k (any of k succeeds) for capability and pass^k (all k succeed) for reliability. [K]
- **Eval the evals.** Seed known-bad and known-good items and confirm the evaluator catches them; use synthetic perturbations (typos, reordered lists) to test robustness. [K]

- **Contamination.** Public benchmarks leak into training data; trust private, product-specific sets more. [K]
- **Per-category reporting.** Report precision/recall per failure category and a confusion matrix, not one blended score. [K]

---

5. Adaptation ladder: prompt to training

Work up the ladder only when the cheaper rung fails: clearer prompt, then examples (few-shot / dynamic few-shot), then retrieval, then tools, then fine-tuning (SFT), then preference optimization or RL. [K]

- **Prompt optimization.** Automated methods (DSPy-style programs, GEPA-style reflective evolution) tune prompts against a metric; they need a trustworthy metric and a held-out set. [K, verify current state]
- **Reasoning models.** Use extended thinking for multi-step problems; avoid over-prescribing step-by-step instructions to models that already reason; set thinking budgets deliberately; measure whether reasoning actually improves your task before paying for it. [K]
- **Prompt versioning and regression.** Store prompts as versioned artifacts; run the gold/regression set on every change. [K]
- **Fine-tuning / distillation.** Good for narrow, high-volume, format-stable tasks (e.g., distill an LLM judge into a small classifier). Data quality beats quantity. Won't reliably inject new facts; use retrieval for that. [K]
- **Preference and RL methods.** DPO-style preference tuning shapes style and behavior; RL with verifiable rewards (RLVR, GRPO-style) improves tasks with checkable answers (code, math, structured tasks). Beware reward hacking. [K]
- **Synthetic data.** Generate, then filter, dedupe, and diversify; verify with a stronger model or code. Training repeatedly on model-generated data risks collapse in diversity. [K]

---

6. Neural network and training intuition

- **Pretraining data.** Quality filtering, deduplication and domain mixture often matter more than raw volume. Repeated epochs on the same data cause memorization and a growing train/validation gap. [K]

- **Scaling laws and data:parameter ratio.** Loss falls predictably with parameters and tokens; compute-optimal ratios exist, and models are often deliberately over-trained (more tokens per parameter) to make inference cheaper. [K]
- **Tokenization.** Subword tokens explain trouble with letter counting, exact character edits and some arithmetic; vocabulary size trades sequence length against embedding size. [K]
- **What post-training changes.** SFT mostly teaches format and behavior; preference tuning shapes style/safety; RL with verifiable rewards can sharpen capability. Distinguish "learned the format" from "learned the skill." [K]
- **Label noise and ambiguity.** Noisy or overlapping labels cap accuracy and hurt calibration; adding an explicit "other" class and cleaning ambiguous classes often helps more than adding data. [K]
- **Data mix interference.** Adding a domain (code, math) shifts performance on other domains; measure per-domain held-out loss. [K]
- **Infrastructure.** MFU (model FLOPs utilization) measures how much hardware compute is used; training is often compute-bound, token-by-token decoding is memory-bandwidth-bound; KV cache, batching, quantization, FlashAttention-style kernels, and speculative decoding explain most inference speed/cost differences. [K]
- **In-context learning.** Examples in the prompt act like temporary training; sensitive to order, label balance and format. [K]
- **Interpretability as intuition.** Probing, attention and activation analysis, and sparse autoencoder features give a feel for what representations exist; useful for intuition, rarely for day-to-day app debugging. [K]
- **Small-model labs.** Karpathy's nanochat provides a full small-LLM pipeline with a single depth dial and a CPU/Apple Silicon path, good for the experiments below. [S]

---

7. Failure-mode catalog (for building taxonomies)

Hallucinated facts; unsupported claims despite provided context (ungrounded); ignored instructions or instruction conflict; format drift; truncation or lazy/partial output; wrong tool or wrong arguments; tool-loop; context overflow or lost-in-the-middle; sycophancy; over-refusal or under-refusal; prompt injection via retrieved/tool content; reward hacking (satisfying the metric not the intent); stale knowledge; inconsistent persona or tone; silent model-version change; judge failures (position, verbosity, self-preference, rubric ambiguity). [K]

---

8. Debugging playbook (decision order)

1. Reproduce on a saved trace; capture exact model version and parameters.
2. Is the needed information in the context? If not, fix retrieval/context.
3. Is the instruction unambiguous and observable? If two readings exist, rewrite or add boundary examples.
4. Is the call making more than one decision? Split it.
5. Ceiling test with a stronger model plus oracle context. Pass means a prompt/layer problem; fail means capability or ambiguous labels.
6. Is the label or judge wrong? Compare with human labels; check kappa and biases.
7. Find the first upstream failure in the trace; fix that layer, not the symptom.
8. Add the failing case to the regression set; confirm the fix does not move other categories.

---

9. Experiment backlog (concept to experiment)

App-side (quick, API-based; run on one of your own gold sets):

- **E1. Binary vs scale.** Same task judged as binary vs 1-5 vs anchored 1-5; compare kappa vs human and run-to-run variance.
- **E2. Schema field order.** Verdict-first vs reasoning-first vs evidence-first; measure accuracy.
- **E3. Label cardinality curve.** Classify with 3, 5, 8, 12, 20 categories; plot accuracy; then test a two-stage hierarchical version.
- **E4. Escape hatch.** With vs without "other/unsure"; measure error on out-of-taxonomy items.
- **E5. Position and verbosity bias.** Pairwise with swapped order; pad answers with filler; measure flip rate.
- **E6. Same-family vs cross-family judge.** Measure self-preference.
- **E7. Jury vs single judge.** Three small diverse models vs one large; cost and kappa.
- **E8. Few-shot curve.** 0, 2, 5, 10 examples; clean prototypes vs boundary cases.
- **E9. Context distractors.** Add irrelevant passages and vary position of the key fact; plot accuracy.
- **E10. Cascade threshold sweep.** Cheap model + escalation on disagreement; cost/accuracy frontier.

- **E11. Calibration.** Verbalized confidence vs logprob vs sample agreement; reliability diagram.
- **E12. Ceiling test.** Run failures through the strongest model with oracle context to separate prompt vs capability failures.
- **E13. Judge distillation.** Fine-tune a small classifier on judge labels (and on noisy vs clean labels); compare to the judge.

Training-side (small models; see the separate opencode prompt): baseline and infrastructure (tokens/sec, MFU), scaling sweep, data noise, data mix, repetition/memorization, tokenization probes, label noise in a classifier, SFT format vs capability.

---

10. Sources found this session (verify; many are vendor blogs)

- Langfuse error analysis: langfuse.com/guides/cookbook/error-analysis-llm-applications and langfuse.com/blog/2025-08-29-error-analysis-to-evaluate-llm-applications
- Hamel Husain evals FAQ (error analysis): hamel.dev/blog/posts/evals-faq/why-is-error-analysis-so-important-in-llm-evals-and-how-is-it-performed.html
- Practitioner evals guide: tianpan.co/blog/2026-02-20-llm-evals-practitioners-guide
- Microsoft scoring whitepaper (June 2026): microsoft.github.io/mcscatblog/assets/posts/better-llm-scoring/20260626_LLM1-5Scoring_WhitePaper.pdf
- Reliability without Validity (21 judges): arxiv.org/pdf/2606.19544
- Judging the Judges (bias mitigation): arxiv.org/pdf/2604.23178
- Judge bias summary: grepture.com/blog/llm-as-a-judge-bias
- Judge reliability on rubric granularity: arxiv.org/abs/2601.08843
- LLM-as-a-Judge survey: awesome-llm-as-a-judge.github.io
- nanochat: github.com/karpathy/nanochat

---

11. Instructions for Opus: turning this into a training schema

Build a curriculum from this document. For each concept, output: (1) a one-line definition, (2) why it matters for building reliable LLM apps, (3) a decision rule or default, (4) one concrete experiment with a pass/fail metric, (5) prerequisites, (6) provenance tag carried

over ([S] or [K]), and (7) a "verify" step for every [K] item (search for the latest evidence, since this field moves quickly). Order into levels: (A) think like the model, (B) decision/schema design, (C) evaluation and judges, (D) pipeline architecture, (E) adaptation ladder, (F) training and infrastructure intuition. Require a written prediction before every experiment and a short post-mortem after. Keep a running "rules of thumb I now trust" table with evidence strength, and flag any rule that fails to replicate on my own data.